C Programming

Assignment 2 (CO2) - Report

Operators, Decision-Making Statements and Looping Constructs

Questions 1 to 4: Precedence and Associativity, Short-Circuit Evaluation, Bitwise Operators, else-if Ladder vs switch

Student name	Sk.Thohith
Register number	191260555
Course / Faculty	
GitHub repository	https://github.com/shaikthohith5353-ship-it/CSA02-1912260555-Assignment-2
Compiler used for measurements	GCC 13.3.0 (Ubuntu 24.04). Compile command: gcc -Wall -Wextra -o qN qN.c
Screenshots	Student's own execution screenshots from an online C compiler (Figures 1 to 5)

Fill in the name, register number, faculty and the GitHub link before submission. The assignment states that a report without a working GitHub link is not evaluated.

Contents and marks

Section	Topic	Marks
Q1	Precedence, associativity and the discount bug (K4)	6
Q2	When does short-circuit evaluation matter? (K4)	4
Q3	Device-status register with bitwise operators, Complete the Code (K4)	6
Q4	else-if ladder or switch? Complete the Code (K4)	6
Appendix	Repository layout, build commands and reproducibility	-

Questions 5 to 7 are not part of this document.

Method used in every section

Each question follows the order required by the assignment: **Predict, Design, Implement, Measure, Analyse, Evaluate**. Predictions were worked out by hand from the C rules before any program was run. Raw counts (function calls, comparisons, loop iterations) are the primary evidence; the screenshots only confirm the final output. Counts are deterministic and can be reproduced exactly; the few timing figures depend on the machine and are used only as supporting observations.

Q1 - Precedence, Associativity and the Discount Bug

Jan Aushadhi Pharmacy billing. 6 marks, Bloom K4 (Analyse).

1.1 Problem analysis

Intended rule: the amount payable is price x quantity, **less 15% senior-citizen discount**, **plus 5% GST**, and then Rs 50 is deducted when the total is above Rs 1000. The assignment does not say which amount GST is charged on. I took the usual billing reading, **GST on the discounted amount**, and the corrected program (Figure 1) uses the same reading.

The developer's code:

```
total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
net = total > 1000 ? total - 50 : total;    /* Rs 50 off above Rs 1000 */
count = count++ + 1;
```

Precedence and associativity rules that apply

Level (high to low)	Operators used here	Associativity
1 (highest)	( )	-
2	* / (same level)	left to right
3	+ - (same level)	left to right
4	>	left to right
5	?:	right to left
6 (lowest)	=	right to left

Precedence decides only *grouping*. It does not fix the order in which the three terms of the sum are evaluated; the compiler may evaluate them in any order, which does not matter here because the terms have no side effects.

1.2 Predictions before execution

Variables were assumed to be **int**, so every / truncates. Three bills were chosen: A (price 200, qty 6, senior), B (price 99, qty 3, senior) and C (price 50, qty 7, not senior).

Operator-by-operator evaluation of the original expression, Bill A (200, 6, senior = 1)

Step	Operation (grouping given by precedence and left-to-right associativity)	Result
1	price * qty (first * of term 1)	1200
2	(price * qty) * 15 (left-to-right, so multiplication before division)	18000
3	18000 / 100	180
4	180 * senior	180
5	(price * qty) * 5, then / 100 (term 3)	6000, then 60
6	1200 - 180 (left-to-right - and +)	1020
7	1020 + 60	1080
8	total = 1080 (assignment is last, lowest precedence)	1080
9	total > 1000, then ?: selects total - 50	1030

Associativity trap: because * and / are left-associative, `price*qty*15/100` multiplies by 15 *before* dividing by 100, which keeps the precision. Had the grouping been `price*qty*(15/100)`, then 15/100 = 0 in integer arithmetic and the discount would vanish (measured: 0).

Bill	Hand prediction, original expression (int)	Hand prediction, double variables	Intended (GST on discounted amount)
A (200, 6, 1)	total 1080, net 1030	1080.00 / 1030.00	gross 1200, discount 180, GST 51, total 1071, net 1021
B (99, 3, 1)	297; 4455/100 = 44; 1485/100 = 14; 297 - 44 + 14 = 267; net 267	297 - 44.55 + 14.85 = 267.30	discount 44.55, GST 12.6225, total 265.07
C (50, 7, 0)	350 - 0 + 1750/100 (= 17) = 367; net 367	350 + 17.5 = 367.50	GST 17.50, total 367.50

I3 Implementation and observed results

Program q1_compare.c evaluates the original expression with int variables, with double variables, and the corrected calculation, for the three bills.

```
#include <stdio.h>
/* Q1: buggy single-expression billing vs corrected, three bills */
typedef struct { int price, qty, senior; } Bill;

int main(void)
{
    Bill b[3] = { {200, 6, 1}, {99, 3, 1}, {50, 7, 0} };
    printf("%-12s %-10s %-10s %-10s %-10s %-10s %-10s\n",
           "Bill(p,q,s)", "int total", "int net", "dbl total", "dbl net", "right tot", "right net");
    for (int i = 0; i < 3; i++) {
        int price = b[i].price, qty = b[i].qty, senior = b[i].senior;

        /* original expression, all-int variables (integer division truncates) */
        int itotal = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
        int inet   = itotal > 1000 ? itotal - 50 : itotal;

        /* original expression, double variables (no truncation, precedence bug stays) */
        double dp = price, dq = qty, ds = senior;
        double dtotal = dp * dq - dp * dq * 15 / 100 * ds + dp * dq * 5 / 100;
        double dnet   = dtotal > 1000 ? dtotal - 50 : dtotal;

        /* corrected: GST on the discounted amount */
        float gross = (float)price * qty;
        float discount = senior ? gross * 15 / 100 : 0;
        float tax = (gross - discount) * 5 / 100;
        float total = gross - discount + tax;
        float net = (total > 1000) ? total - 50 : total;

        printf("(%d,%d,%d)%*s %-10d %-10d %-10.2f %-10.2f %-10.2f %-10.2f\n",
               price, qty, senior, 4, "", itotal, inet, dtotal, dnet, total, net);
    }
    return 0;
}
```

```
Bill(p,q,s)  int total  int net    dbl total  dbl net    right tot  right net
(200,6,1)     1080       1030       1080.00    1030.00    1071.00    1021.00
(99,3,1)     267        267        267.30     267.30     265.07     265.07
(50,7,0)     367        367        367.50     367.50     367.50     367.50
```

Bill	Predicted total/net (int)	Observed int total/net	Observed double total	Intended total/net	Error of original	Verdict
A	1080 / 1030	1080 / 1030	1080.00	1071.00 / 1021.00	+9.00 (Rs 9 overcharged)	WRONG
B	267 / 267	267 / 267	267.30	265.07 / 265.07	+1.93 (int)	WRONG
C	367 / 367	367 / 367	367.50	367.50 / 367.50	-0.50 (int) / 0.00 (double)	wrong in int only

All predictions matched the observed output exactly. The step-by-step trace program q1_steps.c reproduced the Bill A table above:

```
T1=price*qty=1200
T2: price*qty=1200, *15=18000, /100=180, *senior=180
T3: price*qty=1200, *5=6000, /100=60
T1 - T2 = 1020, + T3 = 1080
price*qty*(15/100)*senior = 0  (15/100 is 0 in integer arithmetic)
```

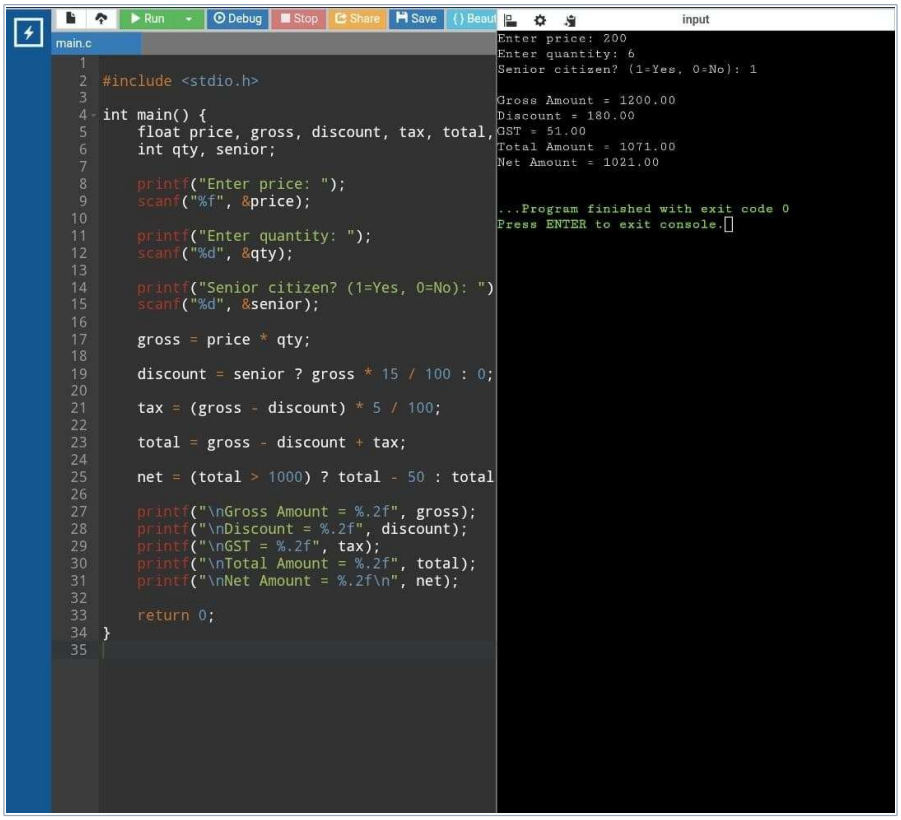


Figure 1 - Student's execution of the corrected program (price 200, qty 6, senior = 1): gross 1200, discount 180, GST 51, total 1071, net 1021. This matches the hand-calculated intended values.

Expressions that differ from what the pharmacist intended

- **GST base (logic error).** The original charges 5% on `price*qty` (the gross amount) instead of the discounted amount. For a senior citizen this overcharges by 5% of the discount: Rs 9 on Bill A (5% of 180).
- **Integer division.** Each `/ 100` truncates before the terms are combined: 4455/100 gives 44 (true 44.55), 1485/100 gives 14 (true 14.85), 1750/100 gives 17 (true 17.50). Money amounts are lost in paise, up to Rs 1 per term.
- **Net amount.** The Rs 50 rule is applied to the wrong total, so the error is carried into `net` (Bill A: 1030 instead of 1021). A bill near Rs 1000 could even be put on the wrong side of the threshold.
- **Third statement.** `count = count++ + 1;` has undefined behaviour (next section).

1.4 Inquiry: why is count = count++ + 1 undefined behaviour?

In this statement the object *count* is modified twice, once by the post-increment and once by the assignment, and its value is also read for the addition. C11 (6.5p2) says that if a side effect on a scalar object is **unsequenced** relative to another side effect on the same object, or to a value computation using that object, the behaviour is undefined. The assignment is sequenced after the value computations of its operands, but nothing orders the store done by ++ against the store done by =. A compiler may therefore produce count+1, produce the old value, or anything else.

What GCC does with -Wall. It compiles the file but prints a warning; it does not stop:

```
q1_count.c: In function 'main':
q1_count.c:6:11: warning: operation on 'count' may be undefined [-Wsequence-point]
    6 |     count = count++ + 1;
      |     ~~~~~~^~~~~~~~~~~~
```

With count = 5, the program printed **count = 6** at -O0, -O2 and -O3 on GCC 13.3. That agreement is only luck of this compiler version; the standard does not guarantee it. The intent is simply to add 1, so the correct statements are `count++;` or `count = count + 1;`.

1.5 Design decision: the corrected calculation

```
gross    = price * qty;
discount = senior ? gross * 15 / 100 : 0;       /* multiply first, divide last */
tax      = (gross - discount) * 5 / 100;        /* the one place that needs brackets */
```

```
total     = gross - discount + tax;
net       = (total > 1000) ? total - 50 : total;
```

Decision: use both intermediate variables and brackets.

- Intermediate variables (gross, discount, tax) give each business rule its own line. Each value can be printed (as Figure I does) and checked against the hand calculation, so the GST-base bug becomes visible.
- Brackets are used in exactly one place, `(gross - discount)`, where the intended grouping differs from what precedence would give. Brackets everywhere would clutter the code; none at all caused the bug.
- The order `gross * 15 / 100` is kept: multiplying before dividing preserves precision.
- Money should not use int division. The submitted program uses float; double (or integer paise) is safer for real billing because float keeps only about 7 significant digits.

1.6 Alternative concept and complexity analysis

Alternative: replace the single long expression by several statements with intermediate variables, which is exactly the corrected program.

Aspect	Single expression (original)	Intermediate variables (corrected)
Arithmetic operations	10 (price*qty is computed three times; the discount term has 4 operators, the GST term 3, the combination 2)	8 arithmetic operations plus one conditional (gross: 1, discount: 2, tax: 3, total: 2)
Time	O(1), 10 operations	O(1), 8 operations. Saves repeated price*qty. With -O2 the compiler usually removes the repetition anyway, so the real saving is small.
Extra space	none declared	3 temporaries (gross, discount, tax) = 12 bytes as float, O(1); normally kept in registers
Correctness	GST base wrong; hard to see	Each step can be tested separately; bug removed
Readability	Needs a precedence table to read	One rule per line, self-documenting

Conclusion: time is O(1) in both versions and the extra space is negligible. Intermediate variables win on correctness and readability, and slightly on operation count.

Pseudocode (corrected)

```
READ price, qty, senior
gross     <- price * qty
IF senior THEN discount <- gross * 15 / 100 ELSE discount <- 0
tax       <- (gross - discount) * 5 / 100
total     <- gross - discount + tax
IF total > 1000 THEN net <- total - 50 ELSE net <- total
PRINT gross, discount, tax, total, net
```